



School of Future Tech

Case Study Report

On

**Autonomous Vehicle Sensor Data Prioritization
System**

by

Aditya khare

Roll No: 150096725061

1. Introduction

Autonomous vehicles must examine nearby objects and quickly decide which ones need immediate attention. This project is a console-based C++ simulation of that prioritization process. It models sensor inputs, combines them through a Sensor Fusion Engine, evaluates risk, uses Quick Sort to rank objects, and produces a decision recommendation.

The report follows the case study architecture while keeping the implementation suitable for a Semester-II Data Structures and Algorithms project. It simulates sensor behavior; it does not connect to real vehicle hardware or control a real car.

2. Objective of the Project

- Simulate LiDAR, Radar, Camera, and GPS readings for nearby road objects.
- Combine the readings using a Sensor Fusion Engine.
- Calculate a risk score automatically instead of accepting a manually entered score.
- Use manual Quick Sort to rank objects from highest risk to lowest risk.
- Display a Priority Ranking Module and a Decision Control recommendation.
- Validate distance, speed, and direction input values.

3. Technologies Used

Area	Details
Programming language	C++17
Core data structure	vector<DetectedObject> and vector<SensorReading>
Primary algorithm	Manual Quick Sort with partition function
Interface	Menu-driven console application
Documentation	Microsoft Word report with architecture and ranking visuals

4. Autonomous Vehicle System Architecture

The project represents the complete architecture requested in the case study. Each sensor module supplies one part of the environmental picture, and the later modules convert that information into a prioritised vehicle response.

Autonomous Vehicle Sensor Data Prioritization Architecture

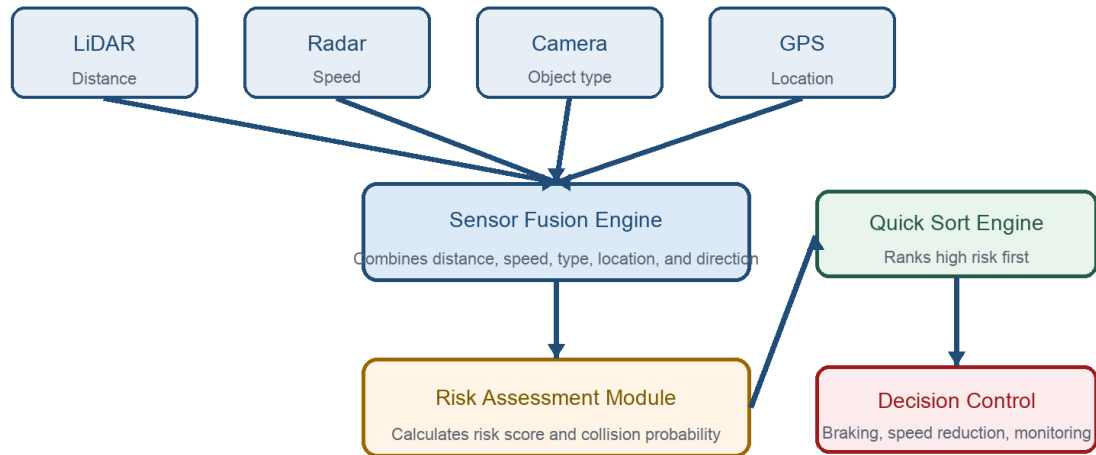


Figure 1: Simulated autonomous vehicle sensor-to-decision architecture.

Module	Project Role
LiDAR	Supplies object distance in metres.
Radar	Supplies relative speed in km/h.
Camera	Identifies object type such as Pedestrian, Dog, Bicycle, Vehicle, or Truck.
GPS	Stores a simulated road-location label.
Sensor Fusion Engine	Combines readings into a DetectedObject.
Risk Assessment Module	Calculates risk using distance, speed, type, direction, and collision probability.
Decision Control System	Recommends braking, speed reduction, steering/lane evaluation, or monitoring.

5. Data Structure Design

Two structures are used. SensorReading represents the raw simulated inputs. DetectedObject is the final fused record that the prioritization engine sorts.

```
struct SensorReading {
    int id;
    string cameraObjectName;
    float lidarDistance;
    float radarSpeed;
    string gpsLocation;
    string direction;
};

struct DetectedObject {
    int id;
```

```

    string objectName;
    float distance;
    float speed;
    int riskScore;
};

```

The vector data structure stores multiple readings and provides index-based access required by the Quick Sort algorithm.

6. Sensor Fusion and Risk Assessment

The Sensor Fusion Engine converts every SensorReading into a DetectedObject. The Risk Assessment Module then assigns a risk score from 0 to 100. It gives more importance to nearby objects, fast-moving objects, vulnerable object types, crossing or approaching movement, and higher predicted collision probability.

Risk Input	How It Affects Priority
LiDAR distance	Shorter distance increases risk.
Radar speed	Higher relative speed increases risk.
Camera object type	Pedestrians, animals, bicycles, and motorcycles receive higher weights.
Movement direction	Crossing and approaching objects receive additional risk.
Collision probability	Estimated from distance, speed, and direction, then included in the final score.

7. Quick Sort Prioritization Engine

Quick Sort is implemented manually through partition() and quickSort(). The last object in the current range is chosen as the pivot. Objects with a greater risk score are moved to the left of the pivot, which produces descending priority order.

```

if (objects[j].riskScore > pivot)
{
    i++;
    swap(objects[i], objects[j]);
}

```

The project does not use std::sort(). This keeps the focus on the Quick Sort data-structures-and-algorithms requirement.

8. Priority Ranking and Decision Control

After Quick Sort, the Priority Ranking Module displays the highest-risk object first. The Decision Control System reads the first object and recommends a basic simulated vehicle action.

Quick Sort Priority Ranking - Sample Sensor Data

Priority	Detected Object	Risk Score	Decision Level
1	Pedestrian	88	Emergency
2	Dog	83	High
3	Vehicle	75	High
4	Truck	67	Medium
5	Bicycle	65	Medium

The highest-risk object is sent to the Decision Control System first.

Figure 2: Sample ranking produced after Sensor Fusion, Risk Assessment, and Quick Sort.

Risk Score	Decision Control Recommendation
85-100	Apply emergency braking and activate collision warning.
65-84	Reduce speed and evaluate steering or lane-change alternatives.
40-64	Maintain safe distance and monitor the object closely.
0-39	Continue normal driving while monitoring surroundings.

9. Program Workflow

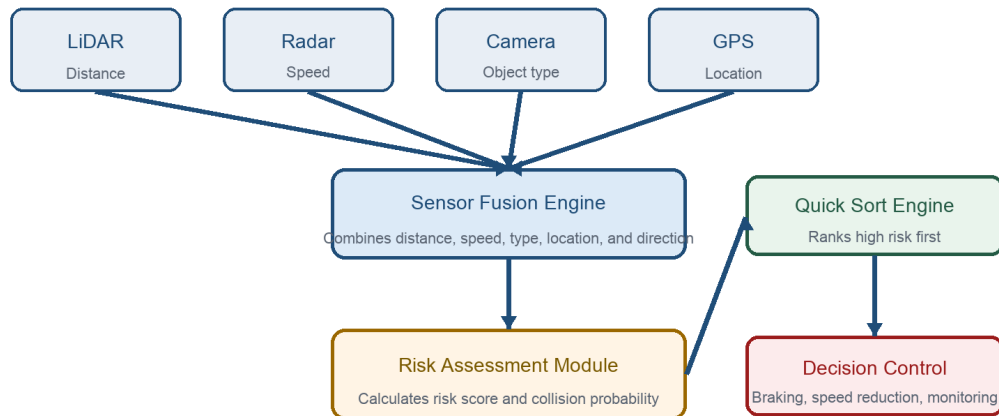
1. Load sample sensor data or capture a new sensor reading.
2. Read LiDAR distance, Radar speed, Camera object type, GPS label, and direction.
3. Run the Sensor Fusion Engine to create detected objects.
4. Run the Risk Assessment Module to calculate risk scores.
5. Apply manual Quick Sort in descending risk-score order.
6. Display the priority ranking and run the Decision Control System.

10. Time and Space Complexity Analysis

Case	Time Complexity
Best case	$O(n \log n)$
Average case	$O(n \log n)$
Worst case	$O(n^2)$

Quick Sort performs in-place partitioning. Its average recursive stack space is $O(\log n)$, so it uses low additional memory compared with algorithms that require a separate array.

Autonomous Vehicle Sensor Data Prioritization Architecture



11. Results and Verification

The completed application was compiled and tested with the sample sensor scenario. The program displayed sensor-layer readings, fused object data, automatically calculated risk scores, sorted the objects manually using Quick Sort, and selected the highest-priority object for a decision recommendation.

Verification Item	Result
LiDAR, Radar, Camera, and GPS modules	Simulated and displayed in the console.
Sensor Fusion Engine	Creates DetectedObject records from sensor readings.
Risk Assessment Module	Calculates risk automatically; manual risk entry is removed.
Quick Sort	Manual implementation; no <code>std::sort()</code> used.
Priority Ranking	Displays highest risk object first.
Decision Control	Recommends an appropriate simulated vehicle action.

12. Limitations

- Sensor values are simulated through sample data or user input; there is no physical sensor hardware.
- GPS is represented by a simple location label instead of real coordinates and maps.

- Collision probability is a simple educational formula, not a machine-learning prediction model.
- Decision outputs are console recommendations and do not control real vehicle actuators.
- The system is designed as an academic DSA demonstration, not a production autonomous-driving system.

13. Conclusion

This project demonstrates how a complete autonomous-vehicle prioritization pipeline can be represented at a Semester-II level. Simulated sensor modules provide the input, Sensor Fusion and Risk Assessment convert readings into meaningful risk scores, and manual Quick Sort prioritizes the objects. The Decision Control System then focuses on the highest-risk object first. The project connects a classical sorting algorithm with a practical autonomous-driving use case while clearly remaining a simulation.

References

1. Case Study 158: Autonomous Vehicle Sensor Data Prioritization System, ITM Skills University.
2. Project source code: Autonomous Vehicle Sensor Data Prioritization System (main.cpp).